Data.Foldable: Ref: https://hackage.haskell.org/package/base/docs/Data-Foldable.html

Data.Foldable

```
class Foldable t where
    {-# MINIMAL foldMap | foldr #-}
    fold :: Monoid m => t m -> m
    foldMap :: Monoid m => (a -> m) -> t a -> m
    foldr :: (a -> b -> b) -> b -> t a -> b
    foldl :: (b -> a -> b) -> b -> t a -> b

foldr1 :: Foldable t => (a -> a -> a) -> t a -> a
foldl1 :: Foldable t => (a -> a -> a) -> t a -> a

length :: Foldable t => t a -> Int
null :: Foldable t => t a -> Bool
elem :: (Foldable t, Eq a) => a -> t a -> Bool
and :: Foldable t => t Bool -> Bool
or :: Foldable t => t Bool -> Bool
all :: Foldable t => (a -> Bool) -> t a -> Bool
any :: Foldable t => (a -> Bool) -> t a -> Bool
concat :: Foldable t => t [a] -> [a]
concatMap :: Foldable t => (a -> [b]) -> t a -> [b]


for_ :: (Foldable t, Applicative f) => t a -> (a -> f b) -> f ()
forM_ :: (Foldable t, Monad m) => t a -> (a -> m b) -> m ()
traverse_ :: (Foldable t, Applicative f) => (a -> f b) -> t a -> f
()
mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()
sequenceA_ :: (Foldable t, Applicative f) => t (f a) -> f ()
sequence_ :: (Foldable t, Monad m) => t (m a) -> m ()
```

Instances

```
instance Foldable Maybe
instance Foldable (Either a)
instance Foldable ((,) a)
instance Foldable []
instance Foldable Identity
...
```

Foldable
Foldable
Typeclassopedia: http://wiki.haskell.org/Typeclassopedia#Foldable

A **Foldable** is a **data structure** which can be **collapsed** i.e. folded over

```
  fold :: Monoid m => f m -> m                    fold summables
  foldMap :: Monoid m => (a -> m) -> f a -> m     fold by mapping to a summable
  foldr :: (a -> b -> b) -> b -> f a -> b         lazy fold     (a0 (a1 (a2 ( ... (an
b) ...)))
```

foldl :: (b -> a -> b) -> b -> f a -> b strict fold (... ((b a0) a1) a2) ...)

A foldable may **short circuit** i.e. terminate early.

```
-- e.g. collapse by summation
foldr (+) 0 [1..10]                -- 55 =  (...
(10+9)+8...+3)+2)+1)
foldMap Sum [1..10]                -- Sum 55 with Data.Monoid.Sum

-- e.g.short circuit
foldr (||) False (repeat True)     -- True = True || (True || ...)
foldMap Any (repeat True)          -- Any True
```

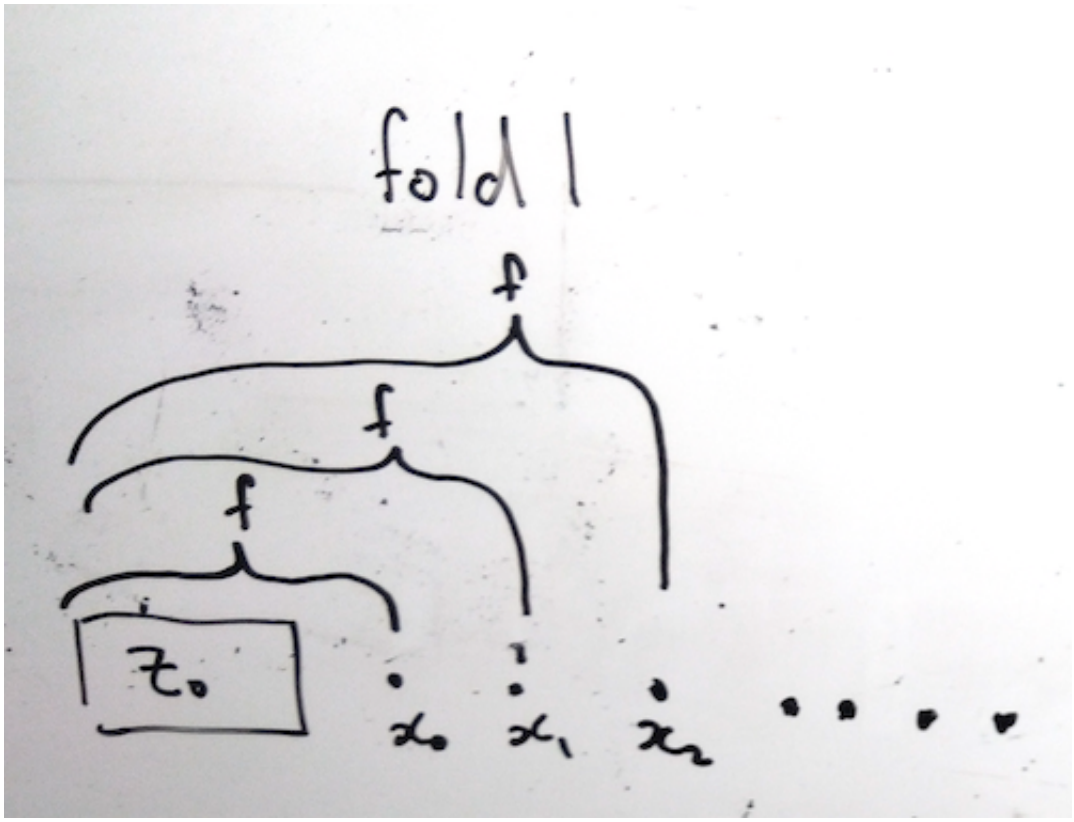
foldl (left associative) vs foldr (right associative)
Ref: https://hackage.haskell.org/package/base/docs/Data-Foldable.html
Ref: http://h2.jaguarpaw.co.uk/posts/foldl-traverses-state-foldr-traverses-anything/

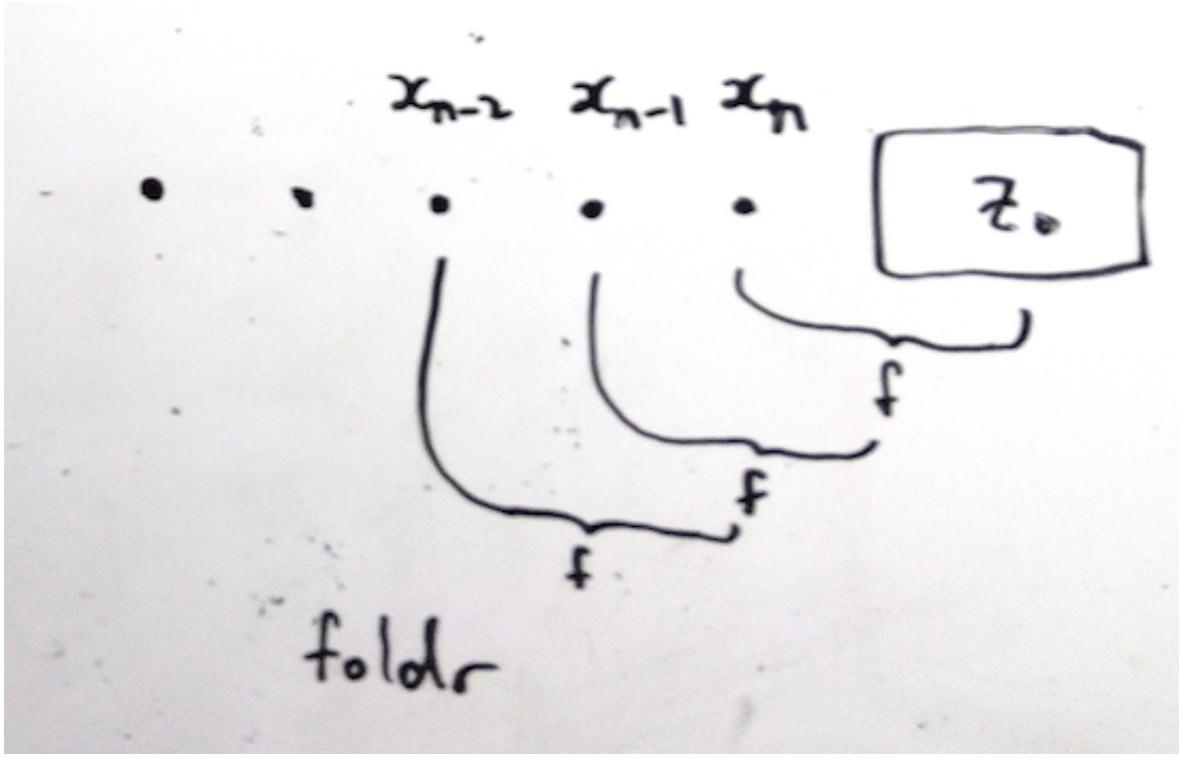
foldl is **left associative** relative to the **base value b**
foldr is **right associative**.

	foldr	**foldl**
Charact erization	**Right** associative aka **FIFO** i.e. first input x0 is also first evaluated in output read i.e. lazy f(x0, ...) **partial** evaluation => f(x0, f(x1, ...)) => f(x0, f(x1, f(x2, ...))) Pseudo code x0 `f` ... => x0 `f` (x1 `f` ... => x0 `f` (x1 `f` (x2 `f` ... => x0 `f` (x1 `f` (x2 `f` ... (xn `f` z) ...))	**Left** associative aka **FILO** i.e. first input x0 is last evaluated on output read i.e. tail recursive f(z, x0) **total** evaluation i.e. => f(f(z, x0), x1) => (f(f(z, x0), x1), x2) ... Pseudo code z `f` x0 => (z `f` x0) `f` x1 => ((z `f` x0) `f` x1) `f` x2) => (... ((z `f` x0) `f` x1) `f` x2) ...) `f` xn

Usage	For **lazy evaluation**:
	e.g. infinite list
	
	`-- Infinite list`
	`ghci> take 2 $ foldr (:) [] [1..]  -- [1,2]`
	`ghci> take 2 $ foldl (flip (:)) [] [1..]  -- never stops`
		For **strict evaluation**:
		e.g. performance
		
		`-- Efficient strict evaluation`
		`ghci> foldl' (+) 0 [1..10^6]`
		`-- fast`
		`ghci> foldr' (+) 0 [1..10^6]`
		`-- slow`

e.g. List	`foldr :: (a -> b -> b) -> b -> [a] -> b`
	`foldr f z [] = z`
	`foldr f z (x:xs) = f x (foldr f z xs)`
		`foldl :: (b -> a -> b) -> b -> [a] -> b`
		`foldl f z [] = z`
		`foldl f z (x:xs) = foldl f (f z x) xs`
		`--  ^ tail recurse`
		`--  i.e. recursion is outer most`
		
		`foldl' :: (b -> a -> b) -> b -> [a] -> b`
		`foldl' f z [] = z`
		`foldl' f z (x:xs) = let z' = f z x`
		`  in z' `seq` (foldl f z' xs)`





e.g. foldl can be implemented by foldr and vice versa
Use **continuation** to invert order of evaluation

```
foldrFromFoldl :: forall a b t. (Foldable t)
  => (a -> b -> b) -> b -> t a -> b
foldrFromFoldl f z0 xs = foldl g id xs z0
 where
  g :: (b -> b) -> a -> (b -> b)
  g k x z = k $ f x z                                -- continuation
k :: b -> b

foldlFromFoldr :: forall a b t. (Foldable t)
  => (b -> a -> b) -> b -> t a -> b
foldlFromFoldr f z0 xs = foldr' g id xs z0
 where
  g :: a -> (b -> b) -> (b -> b)
  g x k z = k $! f z x                               -- continuation
k :: b -> b

-- e.g.
foldr (:) [] [1..10]                        -- [1,2,3..,10]
foldrFromFoldl (:) [] [1..10]               -- same

foldl (flip (:)) [] [1..10]                 -- same
foldlFromFoldr (flip (:)) [] [1..10]        -- same

-- Infinite list
take 2 $ foldr (:) [] [1..]                     -- [1,1]
take 2 $ foldrFromFoldl (:) [] [1..]            -- BAD
```

Usage: fold infinite list

MUST use **foldr**

```
x = foldr (:) [] [1..]
head x
take 10 x

x = foldl (flip (:)) [] [1..]
head x
take 10 x
```

Usage: foldl' (foldl strict)
Use **foldl'**

```
(foldl' (+) 0) [1..10^6])            -- OK. folds outward
                                     --  10^6 + !( ...  !(3 + !(2
+ !(1 + 0))) ... )
(foldr' (+) 0) [1..10^6])            -- BAD. folds inward
                                     -- 1 + !(2 + !(3 + ... ) ... )
```

e.g.

```
ghci> import Criterion.Main

ghci> defaultMain [bench "" (whnf (foldl' (+) 0)
[(1::Int)..10^(6::Int)])]            -- foldl': fast
benchmarking
time                 9.797 ms   (9.786 ms .. 9.809 ms)
                     1.000 R²   (1.000 R² .. 1.000 R²)
mean                 9.795 ms   (9.789 ms .. 9.802 ms)
std dev              17.10 µs   (13.96 µs .. 24.50 µs)

ghci> defaultMain [bench "" (whnf (foldr' (+) 0)
[(1::Int)..10^(6::Int)])]             -- foldr': slow
benchmarking
time                 53.45 ms   (51.98 ms .. 54.82 ms)
                     0.998 R²   (0.997 R² .. 1.000 R²)
mean                 42.50 ms   (35.19 ms .. 46.08 ms)
std dev              9.360 ms   (4.620 ms .. 15.07 ms)
variance introduced by outliers: 77% (severely inflated)

ghci> defaultMain [bench "" (whnf (foldl (+) 0)
[(1::Int)..10^(6::Int)])]                -- foldl: slow
benchmarking
time                 306.3 ms   (256.3 ms .. 359.6 ms)
                     0.975 R²   (0.904 R² .. 1.000 R²)
mean                 267.9 ms   (215.4 ms .. 299.1 ms)
std dev              53.97 ms   (16.48 ms .. 73.55 ms)
variance introduced by outliers: 57% (severely inflated)

ghci> defaultMain [bench "" (whnf (foldr (+) 0)
[(1::Int)..10^(6::Int)])]                -- foldr: slow
benchmarking
time                 166.2 ms   (124.8 ms .. 223.6 ms)
```

```
                     0.902 R²   (0.713 R² .. 0.993 R²)
mean                 133.9 ms   (95.19 ms .. 162.3 ms)
std dev              46.81 ms   (30.72 ms .. 65.37 ms)
variance introduced by outliers: 84% (severely inflated)
```

Foldable utilities
Ref: https://hackage.haskell.org/package/base/docs/Data-Foldable.html

```
-- Library
import Data.Foldable

-- values
elem :: (Foldable t, Eq a) => a -> t a -> Bool
elem v x               -- v in x?
elem 1 [1..10]         -- True
elem -1 [1..10]        -- False


-- concatenation
-- i.e. flattening a list
concat :: Foldable t => t [a] -> [a]
concat
fold                          -- same as fold, restricted to lists.

-- Like flattening a list
concat [[1,2,3],[3,4]]        -- flatten [1,2,3,3,4]
concat $ Just [1,2]           -- [1,2]
concat Nothing                -- []
concat $ Left [1]             -- []
--
concatMap :: Foldable t => (a -> [b]) -> t a -> [b]
concatMap ...
```

==========

Traversable

Data.Traversable: https://hackage.haskell.org/package/base/docs/Data-Traversable.html

Data.Traversable

```
class (Functor t, Foldable t) => Traversable t where
    {-# MINIMAL traverse | sequenceA #-}
    traverse :: Applicative f => (a -> f b) -> t a -> f (t b)
    sequenceA :: Applicative f => t (f a) -> f (t a)
    mapM :: Monad m => (a -> m b) -> t a -> m (t b)
    sequence :: Monad m => t (m a) -> m (t a)

for :: (Traversable t, Applicative f) => t a -> (a -> f b) -> f (t
b)
forM :: (Traversable t, Monad m) => t a -> (a -> m b) -> m (t b)
```

```
forAccumM :: (Monad m, Traversable t) => s -> t a -> (s -> a -> m
(s, b)) -> m (s, t b)
mapAccumL :: forall t s a b. Traversable t => (s -> a -> (s, b)) ->
s -> t a -> (s, t b)
mapAccumR :: forall t s a b. Traversable t => (s -> a -> (s, b)) ->
s -> t a -> (s, t b)
mapAccumM :: forall m t s a b. (Monad m, Traversable t) => (s -> a
-> m (s, b)) -> s -> t a -> m (s, t b)
fmapDefault :: forall t a b. Traversable t => (a -> b) -> t a -> t
b
foldMapDefault :: forall t m a. (Traversable t, Monoid m) => (a ->
m) -> t a -> m
```

Instances

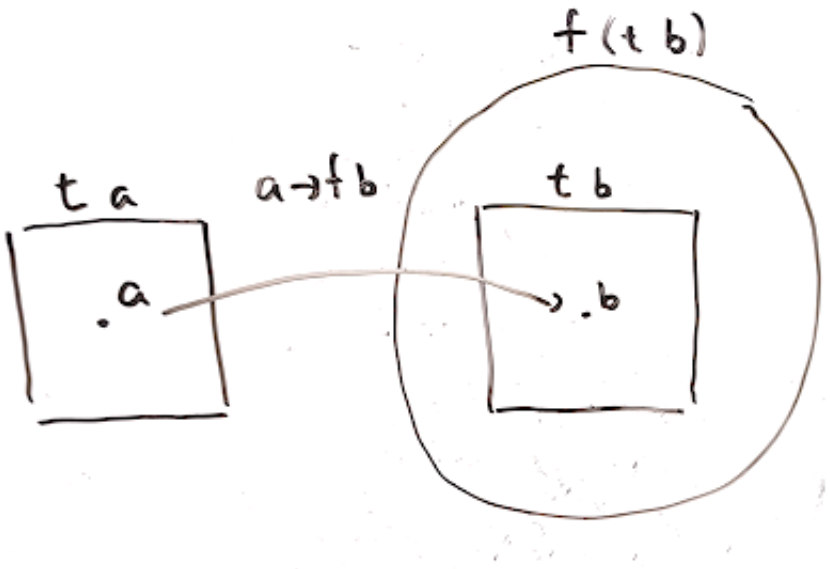
```
instance Foldable Maybe
instance Foldable (Either a)
instance Foldable ((,) a)
instance Foldable []
instance Foldable Identity
...

instance Traversable Maybe
instance Traversable (Either a)
instance Traversable ((,) a)
instance Traversable []
instance Traversable Identity
...
```

Traversable

Traversable
Typeclassopedia: http://wiki.haskell.org/Typeclassopedia
Traversable is a **container (t a)** that can be moved **inside** any **applicative f** as **f (t b)**
a) **traversing i.e. visiting** each value and
b) **reassembling** inside the applicative.

traverse :: (Traversable t, Applicative f) => (a -> f b) -> t a -> f (t b)



Traversable laws
Ref: https://hackage.haskell.org/package/base/docs/Data-Traversable.html

```
t . (traverse f)     == traverse (t.f)             naturality
                                                   applicative
transform i.e. t :: Applicative f, Applicative f' => f a -> f' a
traverse Identity    == Identity                   Identity
                                                   i.e. analogue of
  fmap id == id
Compose . fmap (traverse g) . (traverse f) == traverse (Compose .
(fmap g) . f)
                                                   i.e. analogue of
  (fmap f) . (fmap g) = fmap (f . g)
```

traverse can be seen as a generalization of **map** where the mapping is an applicative
i.e. **Traversable** is a generalization of a **Functor**

```
fmap :: Functor f => (a -> b) -> f a -> f b             -- map with
(a -> b)

traverse :: (Traversable t, Applicative f)              -- map with
(a -> f b)
    => (a -> f b) -> t a -> f (t b)
```

e.g. Print the content of a list
action=IO container=[Int]

```
traverse print [1..10]
```

e.g. Test if all numbers are even
action=success/error container=[Int]

```
justEven x = if even x then Right x else Left (-1)
traverse justEven [0]            -- Right [0]
traverse justEven [0,1]          -- Left (-1)
traverse justEven [0,2]          -- Right [0,2]
```

```
traverse justEven [0,2,3]        -- Left (-1)
traverse justEven [0,2,4]        -- Right [0,2,4]
```

In **implementation**, traverse is defined as
traverse = **lift each value** individually + **lift whole structure** to action via **fold**

```
-- traverse = map + fold
traverse f
sequenceA (f<$>)                -- same. map=f <$>   fold =
sequenceA
-- reverse
sequenceA
traverse id                     -- same.

-- e.g. Traversable []
traverse f list =
 let
   mapped = f <$> list                              step 1) lift each
to action
   append x accum = pure (:) <*> x <*> accum
 in foldr append (pure []) mapped                   step 2) lift
structure to action
```

```
-- Library
import Data.Traversable


---- Basics
--
-- traverse
traverse :: (Traversable t, Applicative f) => (a -> f b) -> t a ->
f (t b)
traverse print [1..10]
traverse (\x -> putStr $ show x) [1..10]
```

```
traverse print [1..10]
1
2
3
...
traverse (\x -> putStr $ show x) [1..10]
1234...
```

```
---- sequenceA: fold
sequenceA :: (Traversable t, Applicative f) => t (f a) -> f (t a)
```

```
---- Traversable examples
justEven x = if even x Just x else Nothing

-- e.g. Traversable []
-- Append to result with (:)
traverse print [1..10]       -- IO [()..] Print each line
traverse justEven [0]       -- Just [0]
traverse justEven [0,1]     -- Nothing
traverse justEven [0,2]     -- Just [0,2]

-- e.g. Traversable (,)
-- Apply on second value
traverse print (0,1)       -- IO (0,()). Prints 1
traverse justEven (0,0)    -- Just (0,0)
traverse justEven (0,1)    -- Nothing

-- e.g. Traversable Maybe
-- Apply only if Just
traverse print $ Just 0     -- IO (Just ()). Prints 0.
traverse print Nothing      -- IO Nothing. Nothing printed.
traverse justEven $ Just 0  -- Just (Just 1)
traverse justEven $ Just 1  -- Nothing
traverse justEven $ Nothing -- Nothing

-- e.g. a Monoid, such as Sum
traverse print $ Sum 1      -- IO (Sum ()). Prints 1
traverse justEven $ Sum 0   -- Just (Sum 0)
traverse justEven $ Sum 1   -- Nothing

-- e.g. Complex
traverse print $ 1:+2      -- IO (Complex ()). Prints 1 2
traverse justEven $ 0:+0   -- Just 0:+0
traverse justEven $ 0:+1   -- Nothing
traverse justEven $ 1:+0   -- Nothing


-- mapM,sequence: same as traverse,sequenceA with Monad instead of
Applicative
mapM :: (Traversable t, Monad m) => (a -> m b) -> t a -> m (t b)
mapM fm x
traverse fm x
-- e.g.
traverse print [1..10]
mapM print [1..10]              -- same
sequenceA $ print <$> [1..10]
sequence $ print <$> [1..10]
```

Foldable & Traversable utilities

```
-- Traversable utilities
--
-- for, forM: traverse flipped
-- In particular, like for loop
```

```
for  :: (Traversable t, Applicative f) => t a -> (a -> f b) -> f (t
b)
forM :: (Traversable t, Monad m) => t a -> (a -> m b) -> m (t
b)            -- restricted to monads
for = flip (traverse)            -- Dfn
forM = flip (mapM)
for x f
forM x f           -- same
traverse f x       -- same
mapM f x           -- same

-- e.g.
for [1..10] print
forM [1..10] print
-- like for loop
for x in [1..10]
print x


-- Traversable *_ utils: analogue of Traversable utils with no
result
-- e.g. traverse vs traverse_
--
-- i.e. output=pure ()
-- Only need Foldable, not Traversable
traverse_ :: (Foldable t, Applicative f) => (a -> f b) -> t a -> f
()
mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()
traverse_ print [1..10]   -- output=pure ()=IO()
mapM_ print [1..10]       -- same
traverse print [1..10]    -- output=IO [(),...]
mapM print [1..10]        -- same

-- sequence
sequenceA_ :: (Foldable t, Applicative f) => t (f a) -> f ()
sequence_ :: (Foldable t, Monad m) => t (m a) -> m ()
sequence_ $ print <$> [1..10]     -- output=pure ()=IO()
sequenceA_ $ print <$> [1..10]    -- same
sequenceA $ print <$> [1..10]     -- output=IO [(),...]
sequence $ print <$> [1..10]      -- same

-- for
for_ :: (Foldable t, Applicative f) => t a -> (a -> f b) -> f ()
forM_ :: (Foldable t, Monad m) => t a -> (a -> m b) -> m ()
for_ [1..10] print     -- output=pure ()=IO()
forM_ [1..10] print    -- same
for [1..10] print      -- output=IO [(),...]
forM [1..10] print     -- same

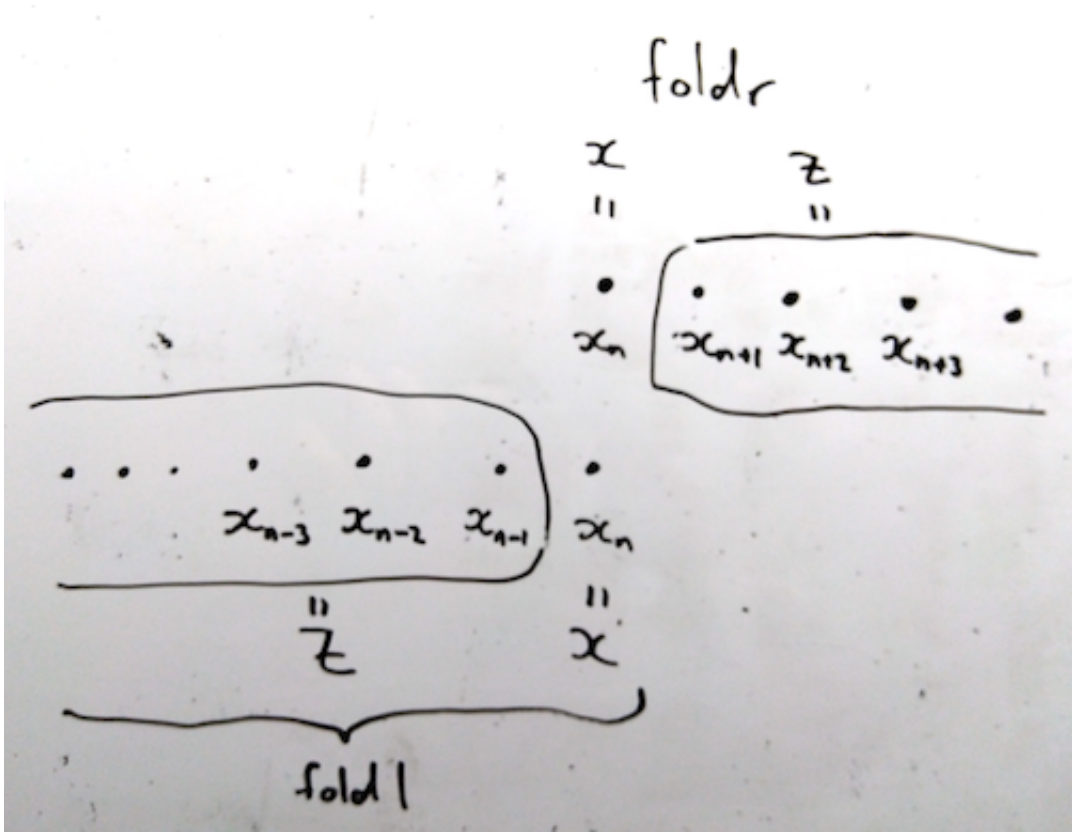

-- mapAccum: like fold + map combined
-- result is
--     (fold, map)
mapAccumR :: Traversable t => (s -> a -> (s, b)) -> s -> t a -> (s,
```

```
t b)
mapAccumL (\accum x->(accum + (2*x), 2*x)) 0 [1..10]   -- (110,
[2,4..20])
mapAccumR (\accum x->(accum + (2*x), 2*x)) 0 [1..10]   -- same, but
uses foldr


---- default functions
-- Same as the usual functions, but using Traversables
--  fmapDefault=fmap
--  foldMapDefault=foldMap
fmapDefault :: Traversable t => (a -> b) -> t a -> t b
fmapDefault (*2) [1..10]          -- [2..20]
fmap (*2) [1..10]                 -- same
foldMapDefault Sum [1..10]        -- Sum 55
foldMapDefault :: (Traversable t, Monoid m) => (a -> m) -> t a -> m
foldMap Sum [1..10]               -- same
```

==========

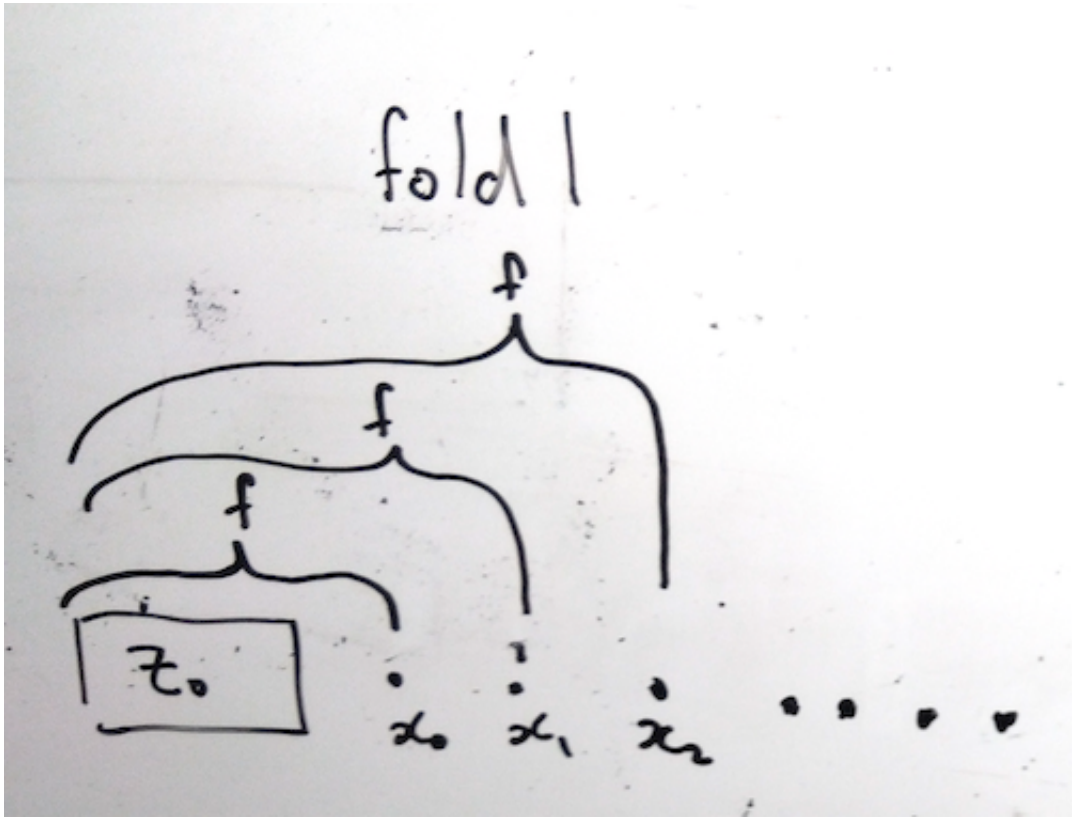
OLD



foldl builds on the **previous values** (apply nth on n-1, n-2, ...), hence **tail recursive**.
foldr builds on the **next values** (apply nth on n+1, n+2, ...)

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl                                   a = [x0, x1, ... xN]
   (\z x -> ... )                       x = x0      z = z0
   z0                                   x = xN      z = result of N-1, N-2, ...
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr                                   a = [x0, x1, ... xN]
  (\x z -> ...)                         x = x0      z = result of 1, 2, ...
  z0                                    x = xN      z = z0
```



	foldr	**foldl**
Charact erization	**Right** associative **FIFO** i.e. first input x0 is also first evaluated in output read i.e. lazy x0 `f` ... => x0 `f` (x1 `f` ... => x0 `f` (x1 `f` (x2 `f` ... => x0 `f` (x1 `f` (x2 `f` ... (xn `f` z) ...))	**Left** associative **FILO** i.e. first input x0 is last evaluated on output read i.e. tail recursive z `f` x0 => (z `f` x0) `f` x1 => ((z `f` x0) `f` x1) `f` x2) => (... ((z `f` x0) `f` x1) `f` x2) ...) `f` xn

List	foldr :: (a -> b -> b) -> b -> [a] -> b foldr f z [] = z foldr f z (x:xs) = f x (foldr f z xs)	foldl :: (b -> a -> b) -> b -> [a] -> b foldl f z [] = z foldl f z (x:xs) = foldl f (f z x) xs -- ^ tail recurse -- i.e. recursion is outer most foldl' :: (b -> a -> b) -> b -> [a] -> b foldl' f z [] = z foldl' f z (x:xs) = let z' = f z x in seq z' (foldl f z' xs)
Usage	-- Infinite list ghci> take 2 $ foldr (:) [] [1..] -- [1,2] ghci> take 2 $ foldl (flip (:)) [] [1..] -- never stops	-- Efficient strict evaluation ghci> foldl' (+) 0 [1..10^6] -- fast ghci> foldr' (+) 0 [1..10^6] -- slow